CampusEats — Services, Contracts & Schema Design

Capabilities

Sl. No.	Capability	Main responsibility
1	Manage Accounts	Registration, login, profiles, campus locations and account information.
2	Manage Catalogue	Restaurants, menus, menu items, prices, availability and operating hours.
3	Manage Orders	Carts, orders, scheduled orders, group orders and order status.
4	Process Payments	Payment authorization, transactions and refunds.
5	Manage Fulfilment	Pickup and campus delivery, assignments and fulfilment status.
6	Manage Engagement	Notifications, ratings and reviews associated with completed orders.

Service Design Benchmark

Service	Responsible member	Data owned
Accounts	Anurag Panda	users, profiles, campus_locations
Catalogue	Dinesh Kushwaha	restaurants, menus, categories, menu_items
Orders	Taj Ansari	carts, cart_items, orders, order_items, group_orders, group_members
Payments	Taj Ansari	payment_methods, payments, transactions, refunds
Delivery	Puspender	deliveries, delivery_assignments, pickup_records
Campus Engagement	Rohit	notifications, ratings, reviews

Cross-service calls:

Caller → Callee	Operation	Purpose
Orders → Accounts	getLocation	Validate/obtain selected campus location.
Orders → Catalogue	checkItem	Validate availability and obtain current item price.
Orders → Payments	charge	Authorize and record payment.
Orders → Delivery	createFulfilment	Create pickup or delivery fulfilment.
Orders → Campus Engagement	sendNotification	Publish an order event notification.
Campus Engagement → Orders	getCompletedOrder	Validate an order before a review/rating.

Boundary rules

- Orders stores its own user reference and does not use a database foreign key into Accounts.
- Orders does not read Catalogue tables; it calls checkItem.
- Orders does not read Payments tables; it calls charge.
- Orders does not write Delivery tables; it calls createFulfilment.
- Campus Engagement does not directly read Orders tables; it uses an order contract.

Service Contracts

A contract exposes operations, inputs, outputs and errors. It does not expose tables, SQL, internal identifier formats, database structure, payment gateways, or implementation details.

Accounts

Operation	Input	Output	Errors
getProfile	userId	profile + account status	not found
getLocation	userId, locationId	campus location	not found, access denied
updateProfile	userId, profile data	updated profile	invalid data

Catalogue

Operation	Input	Output	Errors
listRestaurants	campusId, filter?	restaurants[ ]	invalid campus
getMenu	restaurantId	menu items + prices + availability	restaurant not found
checkItem	itemId	available?, current price	item not found
checkRestaurant	restaurantId	open?, ordering allowed?	restaurant not found

Orders

Operation	Input	Output	Errors
addToCart	userId, itemId, qty	cart	item unavailable, invalid quantity
placeOrder	userId, items, locationId, paymentMethodId, fulfilmentType, scheduledAt?	orderId, status, total, estimatedMinutes	empty order, item unavailable, invalid location, restaurant closed, payment declined, invalid schedule
getOrder	userId, orderId	order + status + items	not found, access denied
cancelOrder	userId, orderId	cancelled status	not found, too late to cancel

Payments

Operation	Input	Output	Errors
charge	userId, amount, paymentMethodId, orderReference	paymentReference, status	declined, invalid method, amount invalid
getPaymentStatus	paymentReference	status + amount	not found
refund	paymentReference, amount	refundReference, status	not found, already refunded, refund not allowed

Delivery

Operation	Input	Output	Errors
createFulfilment	orderId, fulfilmentType, locationId	fulfilmentReference, status	invalid location, unsupported fulfilment
assignDelivery	fulfilmentReference	assignment status	no delivery staff available
getFulfilmentStatus	fulfilmentReference	status + estimated time	not found

Campus Engagement

Operation	Input	Output	Errors
sendNotification	userId, eventType, message data	notificationReference, status	invalid recipient, unsupported event
getCompletedOrder	orderId, userId	reviewable order summary	not found, not completed
createReview	userId, orderId, restaurantId?, itemId?, rating, comment?	reviewReference, status	not eligible, invalid rating, duplicate review

Full Specification of placeOrder

Operation

placeOrder creates an order from a validated set of items and a selected fulfilment option. Orders orchestrates the workflow but does not own catalogue, payment, account-location, or delivery data.

Inputs

Input	Description
userId	Authenticated student requesting the order.
items	[{itemId, quantity}] containing requested menu items.
locationId	Selected campus location for pickup or delivery.
paymentMethodId	Payment method selected by the student.
fulfilmentType	PICKUP or DELIVERY.
scheduledAt?	Optional future time; omitted means immediate ordering.

Outputs

- orderId — reference for the created order.
- status — initial order state.
- total — final amount accepted for the order.
- estimatedMinutes — estimated preparation/fulfilment time.
- fulfilmentType — PICKUP or DELIVERY.

Error cases

Error	Meaning
empty order	No valid items were supplied.
item unavailable	One or more requested items cannot currently be ordered.
price changed	Current Catalogue price differs from the expected price; the order is not silently accepted at an unexpected price.
invalid location	Location is not valid for the selected fulfilment type.
restaurant closed	Restaurant is not accepting orders at the requested time.
invalid schedule	Scheduled time is outside the supported ordering window.
payment declined	Payment authorization failed.
fulfilment unavailable	Requested delivery/pickup option cannot be created.

High-level interaction

- Orders validates the request and obtains the selected location through Accounts.

- Orders checks every item and the current price through Catalogue.
- Orders calculates the accepted order total from Catalogue responses.
- Orders requests payment authorization from Payments.
- If payment succeeds, Orders creates its order state and requests pickup/delivery fulfilment from Delivery.
- Orders requests a notification through Campus Engagement.
- Orders returns the order reference, status, total and estimate to the caller.

What placeOrder hides

- How the order is stored and which internal tables are used.
- How order references are generated.
- How Catalogue stores menus, prices and availability.
- Which payment provider or gateway is used.
- How payment credentials or payment transactions are stored.
- How delivery staff are selected and assigned.
- How notifications are stored and delivered.

Important consistency rule

The client should not send a trusted final price as the authority for the order. Catalogue is the source of truth for current item price and availability. Orders uses the Catalogue contract to validate items before payment. If the price changes, the operation returns an explicit error rather than silently charging an unexpected amount.

Service Validation

Service	Reachable	Self-contained	Contract	Independent	Loosely coupled
Accounts	Yes	Yes	Yes	Yes	Yes
Catalogue	Yes	Yes	Yes	Yes	Yes
Orders	Yes	Yes	Yes	Yes	Yes
Payments	Yes	Yes	Yes	Yes	Yes
Delivery	Yes	Yes	Yes	Yes	Yes
Campus Engagement	Yes	Yes	Yes	Yes	Yes

Validation notes

Reachable: Each service exposes a network API rather than local method access.

Self-contained: Each service owns its listed data and contains the logic needed for its capability.

Has a contract: Each service exposes named operations with defined inputs, outputs and errors.

Independent: No service depends on another service's database tables.

Loosely coupled: Cross-service communication occurs through contracts such as checkItem, charge, getLocation and createFulfilment.